

Attacker & Defender Journal Entries Philosophy

I would also like to inform you about another feature I plan on adding to the book's chapters that I think will really go over well with potential readers:

Attacker/Defender Journal Entries:

Each technical chapter is paired with a first-person journal entry written from the perspective of recurring attacker and defender personas - the same voices threaded throughout the book. On the offensive side, four distinct attacker archetypes (The Opportunist, The Professional, The Ghost, The Insider) narrate their reasoning, tradecraft decisions, and mistakes in real time. ON the defensive side, five practitioner personals (Senior Analyst/ISSO, Junior SOC Analyst, Threat Hunter, IR Lead, Red Team Debriefers) narrate detection triage, and response from their respective vantage points in the security operations lifecycle.

This isn't a stylistic flourish - it is a deliberate pedagogical structure. Security books written in purely academic/technical voice teach what an attack technique does and how to detect but rarely convey why an operator chooses that technique, what it feels like to work a live incident under uncertainty, or how different roles on a SOC team experience the same event differently. The journal format closes that gap. Readers get the mechanics (protocol-level detail, code, KQL/Splunk queries, ATT&CK/D3FEND mappings) in the academic sections, then immediately see those mechanics dramatized through the lens of someone actually making decisions with incomplete information - which is how ethical hacking and defense are practiced in real world environments, not how they're typically packaged in reference texts.

Practically, the journals also do double duty as retention aids: the narrative voice makes the technical content stickier for readers, and the recurring cast gives continuity across chapters so the book reads as a cohesive timeline rather than a disconnected sequence of offensive and defensive techniques. Each entry is scoped tightly to the chapter's technical content (e.g., AS-REP Roasting journals sit alongside the AS-REP Roasting attack/defense chapter/subchapter) so there's no drift between narrative and reference material - the story format is a delivery mechanism for the technical content, not a departure from it.

The first diary "entry" is below so you get an idea of what I mean:

=====

Diaries of an Attacker and a Defender: A Glimpse Inside Both the Attacker and the Defender's Thought Processes for Planning, Executing / Identifying, Detecting, and Preventing an AS-REP Roasting Attack

=====

THREAT ACTOR DOSSIER

Designation:	The Opportunist
Motivation:	Financial / Data Resale on Dark Web

Sophistication: Intermediate
Known Tools: Rubeus, Impacket, Chisel, Custom Python and PowerShell AD
Module LDAP Scripts
OPSEC Discipline: Moderate -- capable, but impatient when under pressure

=====

==

==/=/=/

=====

=====

DEFENDER DOSSIER

Role: Senior SOC Analyst / ISSO Functions
Experience: DoD Network Security Background
Context: Reactive detection, no prior alert on this specific threat vector
Outcome: Full Containment + Proactive Deception Tactic: Honeytoken Deployment

=====

=====

October 25 - 0214

✂ Attacker's Journal Entry

Before I touch this network, I need to develop and lay out a clear attack chain in my notes. I'm thinking through the mechanics of AS-REP roasting, but honestly, I need to first wrap my head around the exact technical foundation of which I'm going to target and exploit first. I know the vulnerability happens when Kerberos pre-authentication is disabled, but I need to map out why that setting even exists and how I can actually break it.

From what I gather, it all comes down to the `userAccountControl` (UAC) attribute in Active Directory. Normally, pre-auth forces a client to encrypt a timestamp using their password hash before the Domain Controller KDC will talk to them. This stops people from just guessing usernames and harvesting hashes offline. But if an account has the `DONT\_REQUIRE\_PREAUTH` flag toggled on - which is 0x400000 in the UAC bitmask - the KDC will completely skip that check. Why does this legacy junk still exist anyway? Oh right...application compatibility - the oldest excuse lazy sysadmins use to not have to deal with fixing or removing those fun exploitable legacy systems they love to keep around... Some ancient garbage that's been sitting on their network since before I was in high school literally cannot handle the modern pre-auth flow, so the admins just... leave it. They just leave the backdoor wide open for me to walk right up in their and pwn everything they've ever set up.

For starters, I need to make sure I don't confuse this mechanically with my upcoming Kerberoasting plans.

My Attack Notes

For AS-REP Roasting:

I don't need any domain credentials to start

I'll just ask the KDC for a ticket for a vulnerable username

The KDC should then hand me back an AS-REP packet containing the encrypted piece I need to obtain the vulnerable target user's own password hash

!! I actually need to make a foothold first before any of this goes down !!:

When I log in (if I'm even able to), I'll have to log in as a low-privileged user depending on what type of vulnerable accounts this poor target may have

Next, I'll request a TGS-REP ticket for a Service Principal Name (SPN), which will give me the hash encrypted with the service account's password

Okay, now that my logic is clear, it's time to list how I'm going to execute this:

First Step:

Reconnaissance

I can't just blindly shoot requests into the dark. I need to enumerate which accounts are misconfigured. I can query their LDAP directly using a custom search filter, or if I already have a shell on a machine, I'll just load up PowerView and run the ``Get-DomainUser -PreauthNotRequired``. The native Active Directory PowerShell module works, too. All I have to do is just find the 0x400000 flag.

Once I have my list of targets organized, it's party time baby: time to extract. From outside their security perimeter, I can feed the usernames I've gathered via my enumerating their domain and port them into Impacket using the ``GetNPUsers.py``. If I'm already inside on one of their Windows machines, I'll just use Rubeus; however, both tools will grab those juicy AS-REP hashes I so desperately need.

But, hold up. I need to be extra careful with OPSEC here because what they're about to experience isn't just a ghost attack. I have to remember that every time I request a ticket, the target's DC might log a Windows Event ID 4768 (Kerberos Authentication Ticket Request). If their internal blue team is actually awake and monitoring their logs, a massive spike of these events all clustered together is going to scream "attacker in the network." I can't just hammer away like some script kiddie. I need to invariably space out my requests, blend in with their background noise of administrative junk that their own systems generate, and slow things the F down.

Then, I gotta think about the encryption downgrade dilemma. I am fully aware that most modern setups default to AES-256, but a lot of environments still support

legacy RC4 (`rc4-hmac`) for backwards compatibility. If I explicitly request an RC4 ticket, it'll give me a hash that will be way faster to crack on my rig. The catch? An isolated RC4 request sticking out like a sore thumb on an all-AES network is an immediate alert trigger waiting to happen for a decent defender who is looking at the encryption type field in Event ID 4768. Do I take the speed or the stealth? I'll have to check and see how their network vibes first.

Once I have captured the hashes and have exfiltrated them to my local machine, it's brute-force party time! I'll need to load them into Hashcat or John the Ripper (JtR). I just need to remember to match the hash format to the correct mode:

```
`18200` = RC4-HMAC  
`19800` = AES-128  
`19900` = AES-256
```

If I get lucky and crack a high-privileged account wide open, that will be my payday. Depending on what privileges any account that I may find holds, they will possibly give me a direct way into their domain, where I can basically, at that point, do anything I want to: achieve domain dominance; laterally move deep into their file servers; use their GPO to deploy this script that will lock out every user and admin account; destroy and pwn their most important file servers and domain controllers (but before I do that - I have to make sure to scan all file shares, check all storage area units and networks, and scan through executive drive spaces for any enticing information I can get my hands on because I know I can sell that type of stuff on the dark web and make a pretty penny - two paydays in one!).

So, my theory is solid and my attack chain seems to be in pretty good order. I'll have @1337H@ck3RKo0! check first before I move forward to make sure I'm not missing anything while also making sure that my OPSEC is good and that I don't get caught (because that means jail time, and I'm not trying to go there)...

Second Step:

Next up, I'm going to draw out a visual kill chain diagram of the Kerberos protocol flow to make sure I don't miss a step when my adrenaline kicks in because I am like a bull in a China tea-glass shop once I'm inside someone's domain...

October 25 - 0245

✂ Attacker's Journal Entry

I'm fleshing out the exact syntax so I don't mess up the typing or forget a switch when I'm live on their target network. I need the exact enumeration queries and the exact Hashcat formatting locked into my notes here.

My Enumeration Commands for Active Directory LDAP

If I manage to drop a beacon or get a PowerShell session on an internal domain-joined machine, I can then use the native AD module or the PowerView tool, which is available for free download!!!!!!!!!!!!!! If I only have to deal with raw

LDAP access from a non-domain asset like my own Linux attack rig via a SOCKS proxy, I can just use a custom LDAP search filter.

Option A: Native PowerShell AD Module

If the native Active Directory module is available on the host I pwn, I can look for the `DoesNotRequirePreAuth` property directly by using this command in PowerShell:

```
Get-ADUser -Filter 'DoesNotRequirePreAuth -eq $true' -Properties  
DoesNotRequirePreAuth | Select-Object Name, SamAccountName, UserAccountControl
```

Option B: PowerView (PowerSploit)

If I need to rely on PowerView to bypass restricted environments or avoid installing malware modules, the built-in wrapper makes it incredibly clean:

```
Get-DomainUser -PreauthNotRequired -Properties samaccountname, useraccountcontrol
```

Option C: Raw LDAP Search Filter (Linux / Proxy Chains)

If I am querying on their domain controller directly from my Linux rig instance using `ldapsearch`, I have to use an extensible match filter to look inside the `userAccountControl` bitmask for 0x400000 (4194304 in decimal) flag:

```
ldapsearch -x -H ldap://<Domain_Controller_IP> -D "user@domain.local" -w  
"Password123" -b "DC=domain,DC=local"  
"(userAccountControl:1.2.840.113556.1.4.803:=4194304)" samAccountName  
userAccountControl
```

My Hashcat Cracking Syntax

Once I grab and pwn target hashes using `GetNPUsers.py` or Rubeus, I can format them differently depending on the encryption type that might be negotiated during my request. I need to make sure I match the correct `hashcat` mode to the signature of the hash output I get. If I'm able to grab accounts that have:

Legacy RC4-HMAC Hashes

Hash Signature: Look for `\$krb5asrep\$23\$` at the beginning of the string

Use command:

```
hashcat -m 18200 -a 0 asrep_rc4.txt /usr/share/wordlists/rockyou.txt -r  
/usr/share/hashcat/rules/best64.rule
```

2. AES-128 Hashes

Hash Signature: Look for `\$krb5asrep\$17\$` at the beginning of the string

Use command:

```
hashcat -m 19800 -a 0 asrep_aes128.txt /usr/share/wordlists/rockyou.txt -r  
/usr/share/hashcat/rules/best64.rule
```

3. Modern AES-256 Hashes

Hash Signature: Look for ` \$krb5asrep\$18\$ ` at the beginning of the string
Use command:

```
hashcat -m 19900 -a 0 asrep_aes256.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

My syntax is the bomb.

Third Step:

Next up, I need to start mapping out a visual for my kill chain process and map out how the AS-REQ and AS-REP network packets will move back and forth between my rig and their KDC.

After that, time to execute. Sayonara, suckers!

Because all ur base are belong to me

October 25 - 0851

✂ Attacker's Journal Entry

Requests are out. The ball is finally starting to roll.

Both of my tools fired clean and as expected - Rubeus handled the targeted pull from my drop-box, no errors, no weird latency spikes on the tunnel. Three accounts came back with the flag set. Three.

Woot.

For a split second there, I was honestly expecting one maybe two if I got lucky. I've already copied the hashes over to my cracking rig via the encrypted channel and fired up and kicked off Hashcat against the first account with rockyou plus best64. RC4 on all three - either their environment never enforced AES or whoever set these accounts up doesn't know jack-shit about jack-shit and is probably still living under a rock stuck in the year 2009. #herpderp

The first hash popped in under just four minutes. Weak ass password. Can't say more in these notes in case this journal ever falls into the wrong hands, like LE or my mom - but, let's just say that I will be having myself a very, very good week, my friends (or enemies).

@1337H@ck3RKo0! replied and gave me a thumb's up on OPSEC. No anomalous spikes on their end watching the perimeter. Traffic blended seamlessly and clean.

Woot x's 2.

Now, I sit on it. Don't touch anything. Don't get greedy. I must maintain composure and keep my wits and focus about me. Let the dust settle before I decide to move

laterally with these sweet cracked creds.

Edit: ...I am absolutely going to get greedy. I already know it. I hate that I know it. I once read a quote that said something to the effect of "when you get greedy is when you get in trouble..." - why I can never follow those words of wisdom is something about myself I guess I'll just never fully understand.

October 25 - 0842

📖 Defender's Journal Entry

I should have just stayed in bed this morning - I woke with the worst headache, and I've been staring at my second cup of coffee for the last ten minutes, trying to decide if what I think I'm chasing is a ghost (another false positive), or network gremlin, or if I'm looking at a real, live breach...

An alert popped up from our SIEM right before my shift started: Event ID 4768. A Kerberos Authentication Ticket Request. Ok...well, normally, that has always seemed to be just standard network background radiation - hundreds of thousands of these fire off every damn day as users log in and machines talk to each other. But... this one was an anomalous cluster that, I think, came from a non-standard workstation asset, and it flagged a specific user account.

My first instinct was to write it off as administrative junk. Some sysadmin probably pushed a broken automation script, or maybe one of our old ass legacy service accounts decided it wanted to contribute to my headache this morning and is misbehaving again. I mean, users forget passwords - we all do; legacy apps retry and retry authentication like there is no tomorrow, and logs spike from time-to-time. And for some reason, we haven't yet been able to connect why this seems to happen every Tuesday. One mentioned that the Accounts and Receiving Department run their monthly funding reports using their in-house software, which we think has a coding issue because it causes their print-server to spit out hundreds of pages that contain nothing but wing-dings-type font. Utter waste of good paper and ink.

I thought about closing the ticket as a false positive (as the ones I have checked in the past, never checked out to be anything of interest...or so I thought).

But then I looked closer at the specific account that triggered it, and my stomach dropped a little - actually, a lot. I took a sip of my lukewarm coffee to ease the dryness I was feeling in my throat, but that didn't help. I noticed the target account the SIEM was complaining about had the `DONT\_REQUIRE\_PREAUTH` flag toggled on in our Active Directory. Looking in my notes, this means that a `0x400000` is active in the User Account Control (UAC) bitmask.

Someone was poking at our AS-REP roasting exposure. Dang it.

October 25 - 0915

📖 Defender's Journal Entry

Okay, my anxiety is officially kicking in now, but it's the defender's flavor of choice: Did I already miss the window? How long has this gap been sitting open for? Has anyone got in? Did anyone on my team notice (because I surely didn't...).

If an attacker is indeed active on this network, that must mean that they have already figured out that this specific account does not require pre-authentication. Mechanically, they didn't even need valid domain credentials to start. They literally just sent a basic `AS-REQ` to our Domain Controller, the KDC saw the legacy flag, completely skipped the timestamp verification check, and blindly handed back an `AS-REP` packet...and inside that packet is a small piece of data that is encrypted with that user's actual domain password hash. The attacker doesn't even need to touch our network anymore for this phase (barring they built a solid attack chain before proceeding with their execution strategies); in fact, if I'm reading into this entire cluster F correctly, the attacker may have already exfiltrated the hash to their C2 infrastructure to attempt to brute-force or use a dictionary/wordlist attack against it offline, where they're free to do whatever with those credentials as they're entirely off our network doing so!

I would like to note for reference that I did check the ticket encryption type field in the log. I noted it as being `0x17` - legacy RC4-HMAC.

My headache just escalated from a 4 to an 8. That 0x17 is a definite red flag - a bad sign because most modern environments will default to using AES (`0x12` or `0x11`). The fact that an RC4 ticket was requested means one of two things:

Either the attacker explicitly forced an encryption downgrade because RC4 hashes crack exponentially faster on a well-built rig with high GPU settings and configuration;

Or, our legacy account is so ancient it doesn't even support AES.

Either way - it means that if they have legit wordlists in hand, they might have already cracked and have them some cleartext credentials to use.

I have decided that this internal argument is over.

This isn't noise.

This is a targeted collection/data harvesting effort.

October 25 - 1030

🕒 Defender's Journal Entry

I thought about going home to try and get rid of this headache, and I thought about doing so without saying anything to anyone about the anomalous events I've noted and witnessed this morning. No one has said anything about it to me. But I can't do that - it's not right, and it's not fair to my team members. I have to friggen fix this! Maybe I'll get some recognition or maybe some time off or a monetary award - who

knows..., and who cares at this point - what matters most at this point for me is figuring out what the hell is going on here - I know I'm onto something - I just don't yet know what...

I'm going to start pivoting from my reactive "I have a bad headache" attitude (because mind over matter, right?), to containment, but I have to move stealthily, quietly, and carefully. I need to know exactly how far or how deep the attacker has gotten into the network before I start flipping switches and tipping my hand to this idiot attacker.

First, I need to hunt down every other account in the domain that has this vulnerability active before the attacker finds them. I'm going to open up PowerShell on the PAW/SAW to run a quick, harmless audit:

```
# Finding every account with Pre-Auth disabled before they do
Get-ADUser -Filter 'DoesNotRequirePreAuth -eq $true' -Properties
DoesNotRequirePreAuth | Select-Object Name, SamAccountName, UserAccountControl
```

Hopefully, the attacker has a beacon or an active shell on the breached endpoint because they might be actively monitoring for process creations or looking for heavy handed admin activity. If I suddenly push a bulk script to re-enable pre-auth across fifty accounts simultaneously, I might spook them - they might run, and before I have any chance to do anything else, knowing my luck, they'll clean tracks and then drop some god-awful ransomware variant or burn their persistence and foothold before I can even locate and attribute the source.

Note: I checked the source IP from the original Event ID 4768. It is tracing back to a DHCP lease assigned to a conference room port. Are they physically in the building?? Or, more likely, are they using a compromised host as a SOCKS proxy???

Before I dig into that, my immediate priorities:

First, I'm going to force a password reset on the compromised account to invalidate whatever they are cracking offline.

I then need to coordinate with the network team to isolate that source IP.

Then, I need to fix this AD misconfiguration in chunks rather than a massive, loud wave.

I will also need to draw up a timeline of the Kerberos requests against our endpoint logs to see exactly when they established persistence and foothold.

I just hope I'm not too late.

October 25 - 1115

🕒 Defender's Journal Entry

My headache is starting to subside. Awesome! Gives me a chance to think more clearly

and logically about what the hell is really going on here...but, my panic is real now.

If they successfully roasted that hash at 0842, and it was a weak RC4 string, if they have a decently set up rig with high GPU processing capabilities, they could have possibly cracked that hash within mere minutes. It's been over two hours since I first noticed the SIEM screaming about Event ID 4768 over and over again. If the attackers already have the cleartext password in hand, I know that by now, they aren't just looking anymore - they're on the move.

I also need to see if they've already used that account to authenticate anywhere else in the domain. I'm pasting my exact SIEM hunting queries so I can keep refreshing them while I wait for the infrastructure team to wake up.

Query 1: Checking for Successful Logons Using the Roasted Account

I need to look for Event ID 4624 (Successful Logon), but specifically I need to look for Logon Type 3 (Network Logon, like accessing a file share) or Logon Type 10 (RDP / Remote Desktop). If I see the roasted account logging into a high-value asset from that same conference room IP, we are officially in incident response mode.

```
index=windows_logs EventCode=4624 TargetUserName="vulnerable_svc_account"
| eval Logon_Type=case(LogonType=3, "Network Share", LogonType=10, "RDP", 1,
"Other")
| table _time, SourceNetworkAddress, TargetMachineName, Logon_Type, Status
```

Query 2: Hunting for the Subsequent TGS Requests

If they didn't do a full interactive login, they might be using that compromised identity to request service tickets (Event ID 4769) for other machines on the domain network.

```
index=windows_logs EventCode=4769
TargetUserName="vulnerable_svc_account@domain.local"
| table _time, IPAddress, ServiceName, TicketEncryptionType, Status
```

October 25 - 1145

🔊 Defender's Journal Entry

So far, no hits on a successful login yet. We might have actually caught this in time, but I can't take any chances. I have to remediate this now, but doing it poorly will only just pull the rug out from under the business. If I just click "Require Pre-Authentication" in Active Directory, whatever critical legacy application is using this account might instantly break or crash, and then I'm the one stuck explaining a self-inflicted outage with a lurking suspected attacker to the board.

My headache just returned after having rationalized all that.

Damn you Murphy! And damn you headache! I must press forward: here is the containment and remediation checklist we drafted up at last quarter's lessons learned tabletop that I'm going to execute right now:

Step 1: Invalidate Existing Tickets (The Nuclear Option)

Even if I change the user's password, any Kerberos Ticket Granting Tickets (TGTs) the attacker has already generated (if any at all since I have yet to see an anomalous albeit successful logon first) might still be valid for the next 10 hours, or so. To completely burn their access, I will first need to reset the domain's `krbtgt` account password, and rotate that thing twice. I know that because of how Kerberos replication actually works, I will have to script the double-reset of the `krbtgt` password to force immediate invalidation of all outstanding tickets across all of our primary and backup AD domain controllers.

Step 2: The Controlled AD Fix

Instead of a blind script, I'm going to target the specific account the SIEM flagged. I'm going to have to force a complex, 32-character password reset first, and then I am manually going to have to uncheck the "Do not require Kerberos preauthentication" box in ADUC.

```
# Flipping the bit back via PowerShell to force Pre-Auth
Set-ADUser -Identity "vulnerable_svc_account" -DoesNotRequirePreAuth $false
```

Step 3: Killing RC4 Globally

This incident proved our legacy fallback as being a massive liability. As soon as this fire is out, I'm writing a proposal to deploy a Group Policy Object (GPO) to explicitly disable RC4 (`0x17`) across the entire domain, forcing everything to use either AES-128 (`0x11`) or AES-256 (`0x12`) - those decisions will have to be made collaboratively with all security-related teams and weigh-in from the ISSO, ISSM, upper leadership, and possibly the CIO, or CISO.

GPO Path (secpol.msc or gpedit.msc): Computer Configuration -> Windows Settings -> Security Settings -> Local Policies -> Security Options -> Network security: Configure encryption types allowed for Kerberos (Uncheck DES and RC4 - check AES).

The network team just located the switch port for that damned conference room IP and effectively disabled it. Phew. The line went dark. Now, I wait for the helpdesk phones to start ringing off the hook - if a legacy app breaks because I turned pre-auth back on, at least we will know exactly which team owns it. But for now, the bleeding has stopped.

October 25 - 1408

📖 Defender's Journal Entry

The smoke has cleared, and the incident response is officially winding down. I'm filling out the final entries in the team wiki and logging the post-incident and

triage notes.

I found out how the attacker got into that conference room port...looking through the physical security logs and cross-referencing them against DHCP leases, someone tailgated an employee into the building at 0815 under the guise of an external AV contractor. They walked straight into the executive boardroom, plugged in an unauthorized device - looks like a pocket-sized Raspberry Pi setup - directly into the exposed floor-box network jack, which was wide open without port-security, and went to town. They were only physically in the room for about twenty minutes before leaving the device to run headlessly. The network team has the physical device now for forensics analysis.

However, I can't just fix this one port and call it a day yet. What if another adversary, or this current attacker's mate gets an internal foothold tomorrow? They'll just end up looking for a different blind spot to hide in, so this means I need to turn this network into a goddamn minefield, and dance toe-to-toe with this attacker who thinks they're going to overrun, overturn, and pwn my network! Challenge accepted!

To ensure we never miss this window again, I am going to deploy a honeypot account - an intentional, highly attractive tripwire designed specifically to catch someone running an AS-REP Roasting or Kerberoasting scan.

The Active Directory Tripwire Setup

I'm documenting the exact steps I used to build this decoy so the rest of the blue team can audit it. The goal here is for me to make this account and tailor it to be enticing and juicy enough to gain the attacker's attention using an automated scanning tool like Rubeus or BloodHound, but is completely fake.

Step 1: Create a Deceptive Identity

To start, I have to create a new user (decoy) account. I'll name it something attractive like `svc-sql-backup`. It sounds like a high-value service account that handles SQL database backups, and if the attacker is smart, will know that the AD database, the ntds.dit file will be contained within these backups, which is exactly the kind of target an attacker is on the hunt for.

Step 2: Configure the Vulnerability: Setting the Bait and Lure

I have deliberately disabled Kerberos pre-auth on this decoy account. I also mapped a fake Service Principal Name (SPN) to it just in case they decide to pivot to Kerberoasting rather than AS-REP Roasting our asses.

Creating the honeypot user account

```
New-ADUser -Name "svc-sql-backup" `
-SamAccountName "svc-sql-backup" `
-Description "MS-SQL Database Backup Service Account" `
-Enabled $true `
-Password (ConvertTo-SecureString "AVeryLongAndComplexPassword123!"
```

```
-AsPlainText -Force)
```

```
# Intentionally exposing it to AS-REP Roasting
```

```
Set-ADUser -Identity "svc-sql-backup" -DoesNotRequirePreAuth $true
```

```
# Adding a fake SPN to bait Kerberoasters too
```

```
Set-ADUser -Identity "svc-sql-backup" -ServicePrincipalNames
```

```
@{Add="MSSQLSvc/sql-prod-db.domain.local:1433"}
```

Step 3: Absolute Isolation

I set a 128-character fully randomized password on this thing. I'll document the math formally later, but the short version is: even with a top-tier GPU cracking rig running flat out, they'd need longer than the universe has existed to brute-force it. It's not getting cracked. Point blank. That's not the point of this account anyway - the point is the alert it generates the second someone or something breathes on it.

The SIEM Alert Rule

Because this account serves absolutely no legitimate business purpose, nobody should ever request a Kerberos ticket for it. If any IP on the network triggers an Event ID 4768 or 4769 against the `svc-sql-backup`, it is a 100% positive indicator of a compromise or an active unauthorized scan.

I just locked this high-fidelity detection query into our SIEM:

```
index=windows_logs (EventCode=4768 OR EventCode=4769)
```

```
TargetUserName="svc-sql-backup"
```

```
| eval Incident_Type=if(EventCode=4768, "AS-REP Roasting Attempt", "Kerberoasting Attempt")
```

```
| stats count by _time, IpAddress, TargetUserName, Incident_Type
```

```
| rename IpAddress as Attacker_IP
```

I hooked this specific alert directly to our automated containment playbook. If this query returns even one single hit, the SIEM will automatically isolate the originating source IP at the switch level within five seconds, then ping my phone.

The next time an attacker tries to map out our Kerberos infrastructure, they aren't going to get their hands on a quiet list of vulnerable targets again. They're going to stomp right onto a landmine, and I'll be waiting for them.

For now, I need another cup of coffee.

October 26 - 0115

✂ Attacker's Journal Entry

I messed up. Bad. I got greedy, and now my heart is hammering against my ribs.

I logged into the C2 panel an hour ago to check the status of my headless drop-box inside the target's executive boardroom. The connection was completely dark. I tried passing traffic through the SOCKS proxy - nothing. Connection timed out.

My first thought was that the battery pack on the Pi died, or maybe some corporate suit unplugged it to plug into a laptop charger, or something. Whatever the case, I needed those AS-REP hashes, so I fired up my backup external beacon to run a quick internal scan to see if the box was still alive on their network. I kept it quiet this time, or at least I thought I did...I targeted a service account I found in my initial passive notes - `svc-sql-backup`. It looked like a standard database service account. I used Rubeus to run a targeted AS-REP roast against just that single username.

The exact millisecond the request fired, my entire external foothold went completely dead.

My terminal threw an immediate connection dropped error. I tried to reconnect via three different redirectors - all burned. They didn't just kill my entire process - they black-holed my entire IP block at the network level!

As I sit here in the dark, the reality of what truly happened is now dawning upon me. That damn `svc-sql-backup` account wasn't ever a true misconfigured legacy account as I initially thought. It was a darn honeytoken. A trap. And I should have known better, but that adrenaline...sheesh, I tell ya. They left pre-auth disabled on purpose to bait me into touching it, and they had an automated playbook hooked up to the alert they configured. The moment my packet hit their KDC asking for that ticket, their SIEM triggered a script of sorts and dropped Thor's hammer on my source IP within seconds.

They now must realize that I've been inside for quite some time now. Worse, they have my physical drop-box. I have to wipe my local staging server, burn these infrastructure keys, and go completely dark before their threat intel team starts tracing these redirectors back to my real-world assets.

Game over for me on this one.

October 26 - 0830

🔒 Defender's Journal Entry

The tripwire went off without a hitch and worked flawlessly and beautifully! At 0112, my phone started screaming. I was still up as I couldn't sleep with my anticipation keeping my adrenaline running on high waiting for the exact moment my phone was to start flipping out being inundated with the configured SIEM alerts I created earlier yesterday.

It finally happened. The SIEM caught a rogue Event ID 4768 targeting my fake `svc-sql-backup` identity. The automated incident response playbook didn't hesitate - it identified the source IP, traced it back to an external VPN endpoint we hadn't yet mapped, and dynamically updated the perimeter firewall blocklists to completely

isolate the threat actor within four mere seconds.

Meanwhile, the hardware forensic guys and gals just finished analyzing the confiscated Raspberry Pi device that was pulled from the executive boardroom floor-box. I just pasted their raw teardown notes into my journal.

Forensic Breakdown: Captured Unauthorized Rogue Asset

- Hardware Profile

Raspberry Pi 4 Model B (4GB RAM) concealed inside a generic, unbranded black plastic case; however, a laminated sticker of "Lenny, shrug-face emoticon (`_('ヅ) _/')" was displayed on the front of the device's black encasement. Powered by an inline 20,000mAh external battery pack, tucked flat underneath the floor-box cable routing tray.

- Physical Connectivity

Connected directly to the boardroom network drop via flat, black CAT6 ribbon cable to minimize visual detection. A secondary USB WiFi dongle (Alfa AWUS036ACM) was attached, likely intended for local wireless exfiltration if the internal network interface got locked down. Used as a persistence mechanism for future or immediate access back into specific legs (portions) of the network.

Storage & Logic Analysis (SD Card Forensic Image) Using FTKImager and the Sleuth Toolkit

The Digital Forensics & Incident Response (DFIR) team pulled the MicroSD card, mounted the EXT4 partition write-blocked, and ran a full artifact analysis against it. The findings gathered are detailed below:

- Operating System (OS) Environment: Running a heavily stripped-down, headless version of Arch Linux Arm.

- Attacking C2 Infrastructure Configuration: The device was running a lightweight Chisel binary that was configured to establish a reverse TCP tunnel over port 443 (`https`) out to an external cloud VPS. By disguising the malicious Command and Control (C2) traffic as standardized outbound web browsing traffic, the attacking entity was able to bypass our basic egress firewall rules.

Scripting & Toolkit

- Discovered a custom Python script in `/opt/scout/` designed to automatically parse internal Active Directory information using anonymous LDAP binds.

- Found a cached flat-text file that contained the usernames they had successfully enumerated during their short window on Tuesday morning - including the target account from the original SIEM alert.

The Smoking Gun

The most satisfying part? The attacker's local bash history file (`.bash\_history`) wasn't linked to `/dev/null`. We can see the exact, panicked syntax they typed right before we killed their port access on Tuesday, including their manual verification checks against our Domain Controllers.

We have since handed the external IP addresses, the Chisel server configurations, and the physical artifacts over to corporate legal and external law enforcement. We went from reactive panic forty-eight hours ago to an ironclad, proactive defense.

Now, I'm going home to put the fire out in my head that has been this headache for almost two days. At least this time, I know I'll be able to rest and recover with peace of mind knowing our defense infrastructure is more secure than it was two days ago.
